

Implementación de un sistema de detección de intrusiones con notificación de alertas en tiempo real

Suricata 7.0.10 sobre Debian 13 (Trixie)

Despliegue, escritura de firmas propias e integración con el escritorio

Gonzalo Rodríguez

Analista en Sistemas — Especialista en Ciberseguridad

github.com/GonzaBot

Rivera, Uruguay — 31 de agosto de 2026

Índice

Índice.....	2
1. Introducción.....	3
1.1. Objetivo.....	3
1.2. Alcance y limitaciones.....	3
2. Entorno de trabajo.....	3
3. Instalación.....	4
3.1. Instalación del paquete.....	4
3.2. Verificación de la compilación.....	4
4. Configuración.....	5
4.1. Diagnóstico del fallo inicial.....	5
4.2. Definición de la red protegida.....	6
4.3. Interfaz de captura.....	6
5. Gestión de reglas.....	7
5.1. Descarga del conjunto de firmas.....	7
5.2. Validación de la configuración.....	8
6. Puesta en servicio.....	9
6.1. Activación.....	9
6.2. Verificación de captura.....	10
7. Reglas personalizadas.....	11
7.1. Firma de detección por contenido.....	11
7.2. Error de parseo y corrección.....	12
7.3. Verificación de la detección.....	13
7.4. Firma de detección de escaneo.....	14
8. Notificación de alertas en el escritorio.....	15
8.1. Consideración arquitectónica.....	15
8.2. Verificación del canal de notificaciones.....	15
8.3. Script de vigilancia.....	16
8.4. Servicio de usuario.....	17
8.5. Verificación funcional.....	18
9. Prueba integral.....	19
10. Mantenimiento.....	20
11. Conclusiones.....	20
11.1. Líneas de trabajo posteriores.....	20
Anexo. Inventario de archivos.....	21

1. Introducción

Este trabajo documenta el despliegue completo de Suricata como sistema de detección de intrusiones sobre una estación de trabajo Debian 13, desde la instalación del paquete hasta la notificación visual de alertas en el escritorio. El recorrido incluye la corrección de dos fallos reales que surgieron durante la implementación, la escritura de firmas propias y la resolución de un problema de arquitectura vinculado a los permisos de sesión gráfica.

La elección de documentar también los errores es deliberada. Un procedimiento que solo muestra comandos exitosos oculta justamente el conocimiento que hace falta cuando algo se rompe en producción, y los dos fallos que aparecen en este informe ilustran conceptos centrales del motor: la orientación de las reglas respecto de la red protegida y el funcionamiento de los buffers de inspección.

1.1. Objetivo

Poner en funcionamiento un IDS que analice el tráfico de red del equipo en tiempo real, lo contraste contra un conjunto de firmas de amenazas conocidas, y notifique al usuario en pantalla ante cualquier evento sospechoso, sin necesidad de que exista una terminal abierta ni intervención manual tras el inicio de sesión.

1.2. Alcance y limitaciones

El equipo cuenta con una única interfaz de red inalámbrica operando en modo administrado. Esto determina el alcance real del sistema: Suricata inspecciona el tráfico que entra y sale del propio equipo, no el del resto de los dispositivos de la red. Se trata, en rigor, de detección a nivel de host mediante un motor de red.

Extender la cobertura a toda la red requeriría una de tres alternativas: configurar un puerto espejo en el switch, ubicar el equipo como pasarela del segmento, o pasar la interfaz inalámbrica a modo monitor, opción esta última que inhabilita la conectividad normal. Ninguna se aplicó en este despliegue, y la limitación se declara de forma explícita porque condiciona la interpretación de los resultados.

Un segundo límite, propio del contexto actual de la web, es que buena parte del tráfico viaja cifrado. Durante las pruebas se observó que la mayoría de los eventos capturados correspondían al protocolo QUIC, es decir HTTP/3 sobre UDP. En esos casos el motor registra el flujo y sus metadatos, pero no puede inspeccionar el contenido.

2. Entorno de trabajo

El despliegue se realizó sobre un equipo de escritorio con Debian 13.6 (Trixie) y entorno KDE Plasma sobre Wayland. La interfaz activa es un adaptador inalámbrico USB identificado como wlxc83a35cc2045, con dirección 192.168.1.100 dentro del segmento 192.168.1.0/24. El equipo dispone de 15 GB de memoria y cuatro núcleos de procesamiento.

La versión de Suricata instalada desde los repositorios oficiales es la 7.0.10, y el conjunto de reglas proviene de Emerging Threats Open. Como herramientas complementarias se emplearon jq para el procesamiento de JSON, libnotify para las notificaciones de escritorio y nmap para la validación final.

3. Instalación

3.1. Instalación del paquete

Suricata se encuentra disponible en los repositorios oficiales de Debian, de modo que la instalación no requiere agregar fuentes externas. El paquete arrastra un conjunto considerable de dependencias, entre ellas las bibliotecas de DPDK, hyperscan para la búsqueda de patrones y libhttp para el análisis del protocolo HTTP.

```
sudo apt update
sudo apt install suricata -y
```

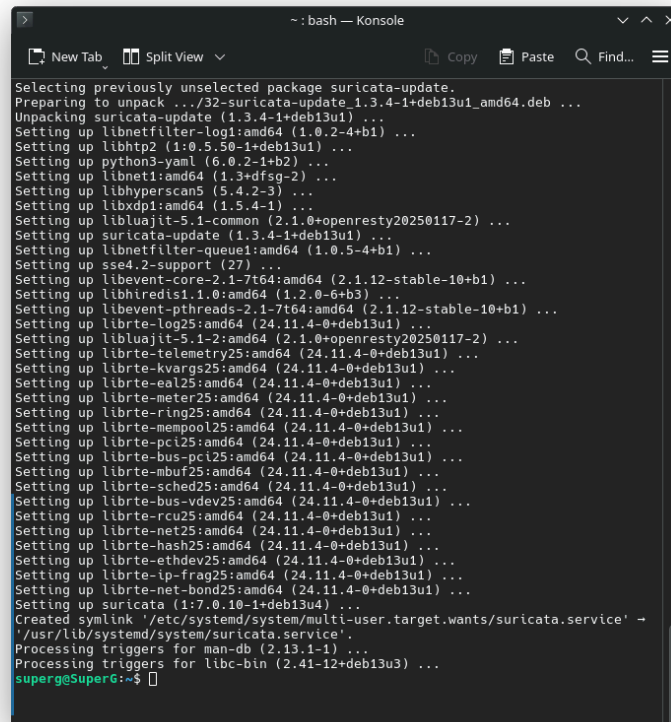
A terminal window titled '~: bash — Konsole' showing the output of the command 'sudo apt install suricata -y'. The output lists numerous dependencies being installed, including libnetfilter-log1, libhttp2, python3-yaml, libnet1, libhyperscan5, libxdp1, liblua5.1-common, libsuricata-update, libnetfilter-queue1, libevent-core, libhiredis1, libevent-pthreads, librt-log25, liblua5.1-2, librt-telemetry25, librt-kvargs25, librt-eal25, librt-meter25, librt-ring25, librt-mempool25, librt-pci25, librt-bus-pci25, librt-mbuf25, librt-sched25, librt-bus-vdev25, librt-rcu25, librt-net25, librt-hash25, librt-ethdev25, librt-ip-frag25, librt-net-bond25, and finally suricata. The terminal also shows the creation of a symlink for the systemd service and the processing of triggers for man-db. The prompt at the bottom is 'super@Super6:~\$'.

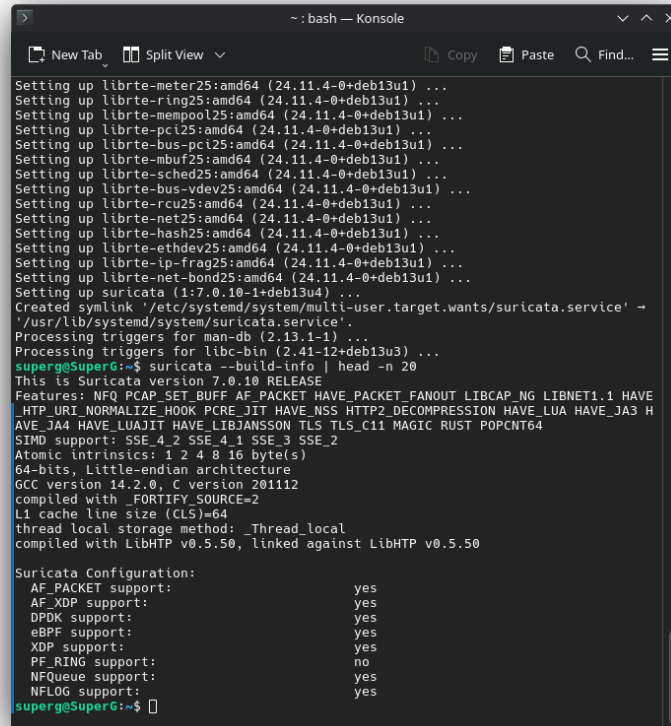
Figura 1 — Instalación del paquete y creación automática del servicio systemd

Al finalizar, el instalador registra el servicio en systemd y lo habilita para el arranque, según se aprecia en la creación del enlace simbólico hacia multi-user.target.wants.

3.2. Verificación de la compilación

Antes de configurar conviene confirmar con qué características fue compilado el binario, ya que determinan los modos de captura disponibles.

```
suricata --build-info | head -n 20
```



```
Setting up librt-meter25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-ring25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-mempool25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-pci25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-bus-pci25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-mbuf25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-sched25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-bus-vdev25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-rcu25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-net25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-hash25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-ethdev25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-ip-frag25:amd64 (24.11.4-0+deb13u1) ...
Setting up librt-net-bond25:amd64 (24.11.4-0+deb13u1) ...
Setting up suricata (1:7.0.10-1+deb13u4) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/suricata.service' ->
'/usr/lib/systemd/system/suricata.service'.
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for libc-bin (2.41-12+deb13u3) ...
super@SuperG:~$ suricata --build-info | head -n 20
This is Suricata version 7.0.10 RELEASE
Features: NFQ PCAP_SET_BUF AF_PACKET HAVE_PACKET_FANOUT LIBCAP_NG LIBNET1.1 HAVE
_HTTP_URI_NORMALIZE_HOOK PCRE_JIT HAVE_NSS HTTP2_DECOMPRESSION HAVE_LUA HAVE_JA3 H
AVE_JA4 HAVE_LUAJIT HAVE_LIBJANSSON TLS TLS_C11 MAGIC RUST POPCNT64
SIMD support: SSE_4_2 SSE_4_1 SSE_3 SSE_2
Atomic intrinsics: 1 2 4 8 16 byte(s)
64-bits, Little-endian architecture
GCC version 14.2.0, C version 201112
compiled with _FORTIFY_SOURCE=2
L1 cache line size (CLS)=64
thread local storage method: _Thread_local
compiled with LibHTP v0.5.50, linked against LibHTP v0.5.50

Suricata Configuration:
AF_PACKET support: yes
AF_XDP support: yes
DPDK support: yes
eBPF support: yes
XDP support: yes
PF_RING support: no
NFQueue support: yes
NFLOG support: yes
super@SuperG:~$
```

Figura 2 — Versión instalada y características de compilación

La salida confirma soporte para AF_PACKET, AF_XDP, DPDK y eBPF. El modo relevante para este despliegue es AF_PACKET, que es además el que utiliza la unidad de systemd que trae el paquete de Debian.

4. Configuración

4.1. Diagnóstico del fallo inicial

Tras la instalación el servicio no logró iniciar. La consulta a systemd informaba únicamente un código de salida 1 acompañado de reintentos sucesivos, sin indicar la causa. Esto ocurre porque Suricata arranca en modo demonio y desacopla su salida del proceso padre, de modo que el motivo del fallo queda registrado en su propio archivo de registro y no en el diario del sistema.

```
sudo tail -n 20 /var/log/suricata/suricata.log
```

La revisión de ese archivo reveló dos problemas simultáneos. El primero, causa directa de la caída, era que la configuración por defecto apuntaba a una interfaz llamada eth0 inexistente en este equipo. El segundo, más silencioso, indicaba que ningún archivo de reglas coincidía con el patrón esperado, de manera que el motor se habría iniciado sin una sola firma cargada.

Esta segunda condición merece atención: un motor sin firmas arranca correctamente y se reporta como activo, pero no detecta absolutamente nada. Es un modo de fallo que no genera error visible.

4.2. Definición de la red protegida

La variable `HOME_NET` define qué direcciones considera propias el motor. Su valor por defecto abarca la totalidad del espacio de direccionamiento privado, y se restringió al segmento real en uso.

```
vars:
  address-groups:
    HOME_NET: "[192.168.1.0/24]"
    EXTERNAL_NET: "!$HOME_NET"
```

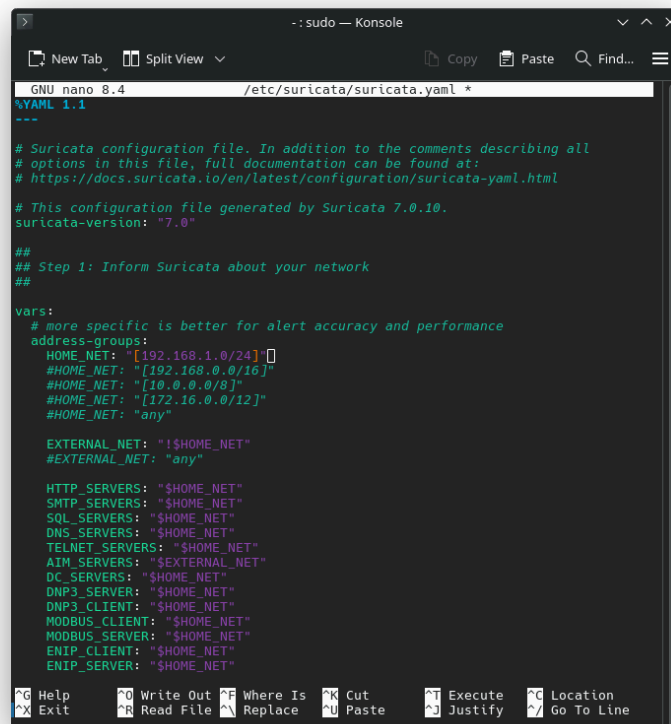


Figura 3 — Definición de `HOME_NET` restringida al segmento real

El ajuste tiene consecuencias prácticas mayores de lo que sugiere su simplicidad. Las firmas de Emerging Threats están redactadas en términos de dirección del flujo, típicamente desde la red externa hacia la protegida. Si `HOME_NET` abarca todo el espacio privado, tráfico que debería clasificarse como externo queda considerado interno y un conjunto amplio de reglas deja de activarse. El sistema funciona, reporta estado correcto, y no detecta.

4.3. Interfaz de captura

El segundo ajuste corrige la causa del fallo de arranque, indicando al bloque af-packet la interfaz real del equipo.

```
af-packet:
- interface: wlxc83a35cc2045
  cluster-id: 99
  cluster-type: cluster_flow
```

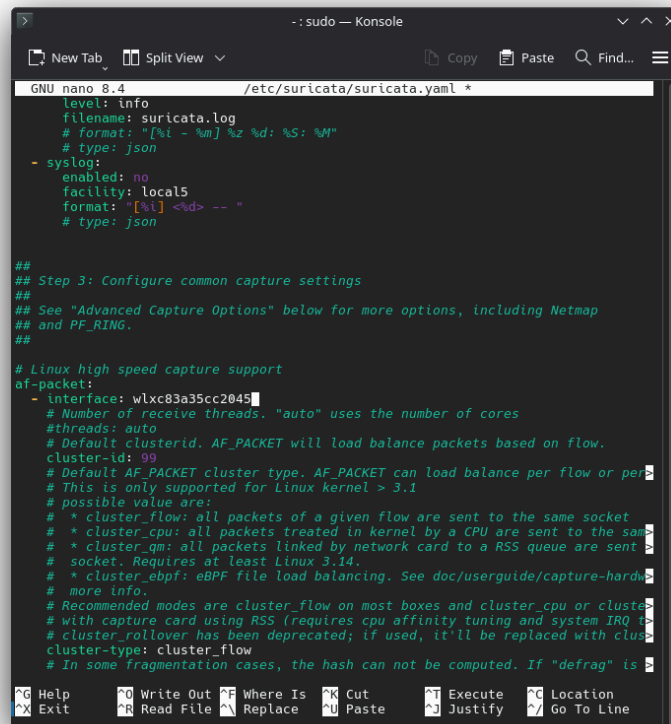


Figura 4 — Interfaz de captura corregida en el bloque af-packet

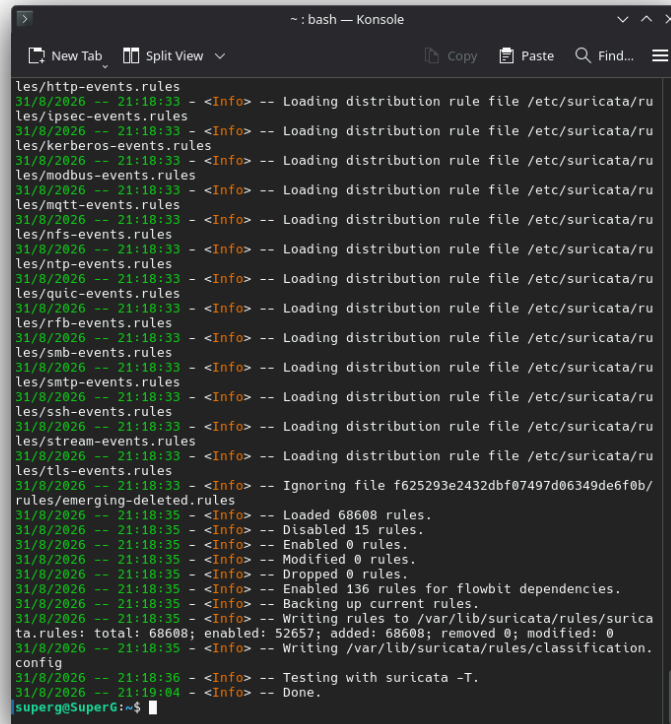
El archivo de configuración contiene otras apariciones de eth0 en las secciones correspondientes a pcap, netmap, af-xdp y dpdk. Se dejaron sin modificar, dado que el motor únicamente interpreta la sección del modo de captura con el que fue invocado. Conservarlas evita introducir cambios innecesarios en un archivo extenso.

5. Gestión de reglas

5.1. Descarga del conjunto de firmas

La utilidad `suricata-update` descarga y consolida el conjunto Emerging Threats Open, mantenido de forma gratuita y actualizado con regularidad.

```
sudo suricata-update
```



```
~: bash — Konsole
New Tab Split View Copy Paste Find...
les/http-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/ipsec-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/kerberos-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/modbus-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/mqtt-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/nfs-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/nntp-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/quic-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/rfb-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/smb-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/smtp-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/ssh-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/stream-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/tls-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Ignoring file f625293e2432dbf07497d06349de6f0b/
rules/emerging-deleted.rules
31/8/2026 -- 21:18:35 - <Info> -- Loaded 68608 rules.
31/8/2026 -- 21:18:35 - <Info> -- Disabled 15 rules.
31/8/2026 -- 21:18:35 - <Info> -- Enabled 0 rules.
31/8/2026 -- 21:18:35 - <Info> -- Modified 0 rules.
31/8/2026 -- 21:18:35 - <Info> -- Dropped 0 rules.
31/8/2026 -- 21:18:35 - <Info> -- Enabled 136 rules for flowbit dependencies.
31/8/2026 -- 21:18:35 - <Info> -- Backing up current rules.
31/8/2026 -- 21:18:35 - <Info> -- Writing rules to /var/lib/suricata/rules/surica
ta.rules: total: 68608; enabled: 52657; added: 68608; removed 0; modified: 0
31/8/2026 -- 21:18:35 - <Info> -- Writing /var/lib/suricata/rules/classification.
config
31/8/2026 -- 21:18:36 - <Info> -- Testing with suricata -T.
31/8/2026 -- 21:19:04 - <Info> -- Done.
superG@SuperG:~$
```

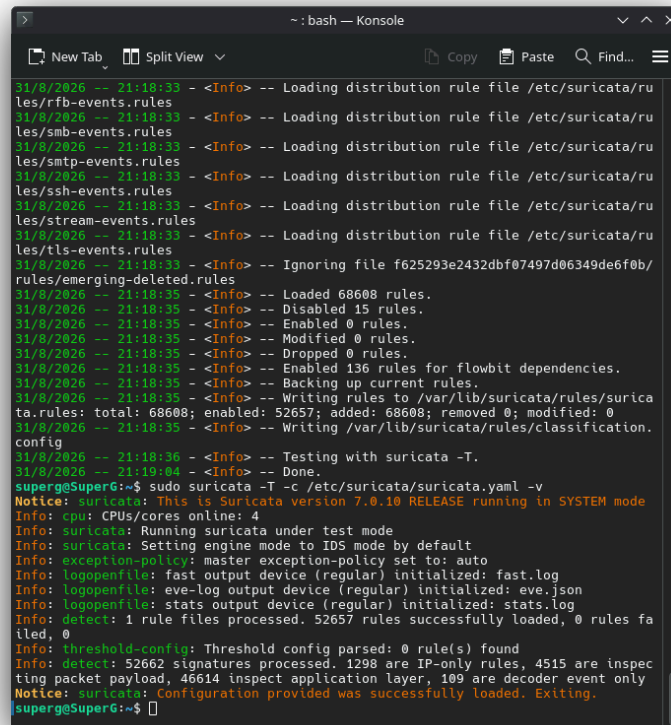
Figura 5 — Descarga y consolidación del conjunto Emerging Threats Open

El proceso incorporó 68.608 reglas, de las cuales quedaron habilitadas 52.657. La diferencia corresponde a firmas desactivadas por defecto, en general por su tendencia a producir falsos positivos en entornos comunes. La herramienta habilitó además 136 reglas adicionales para resolver dependencias de flowbits, mecanismo por el cual una firma marca un estado del flujo que otra consulta posteriormente.

5.2. Validación de la configuración

Suricata dispone de un modo de prueba que carga la configuración y las firmas sin iniciar la captura. Conviene ejecutarlo antes de cada reinicio del servicio, ya que informa los errores de manera directa en lugar de dejarlos sepultados en el registro del demonio.

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```



```
~: bash — Konsole
New Tab Split View Copy Paste Find...
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/rfb-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/smb-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/smtp-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/ssh-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/stream-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Loading distribution rule file /etc/suricata/ru
les/tls-events.rules
31/8/2026 -- 21:18:33 - <Info> -- Ignoring file f625293e2432dbf07497d06349de6f0b/
rules/emerging-deleted.rules
31/8/2026 -- 21:18:35 - <Info> -- Loaded 68608 rules.
31/8/2026 -- 21:18:35 - <Info> -- Disabled 15 rules.
31/8/2026 -- 21:18:35 - <Info> -- Enabled 0 rules.
31/8/2026 -- 21:18:35 - <Info> -- Modified 0 rules.
31/8/2026 -- 21:18:35 - <Info> -- Dropped 0 rules.
31/8/2026 -- 21:18:35 - <Info> -- Enabled 136 rules for flowbit dependencies.
31/8/2026 -- 21:18:35 - <Info> -- Backing up current rules.
31/8/2026 -- 21:18:35 - <Info> -- Writing rules to /var/lib/suricata/rules/surica
ta.rules: total: 68608; enabled: 52657; added: 68608; removed 0; modified: 0
31/8/2026 -- 21:18:35 - <Info> -- Writing /var/lib/suricata/rules/classification.
config
31/8/2026 -- 21:18:36 - <Info> -- Testing with suricata -T.
31/8/2026 -- 21:19:04 - <Info> -- Done.
superG@SuperG:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 4
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 1 rule files processed. 52657 rules successfully loaded, 0 rules fa
iled, 0
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 52662 signatures processed. 1298 are IP-only rules, 4515 are inspec
ting packet payload, 46614 inspect application layer, 109 are decoder event only
Notice: suricata: Configuration provided was successfully loaded. Exiting.
superG@SuperG:~$
```

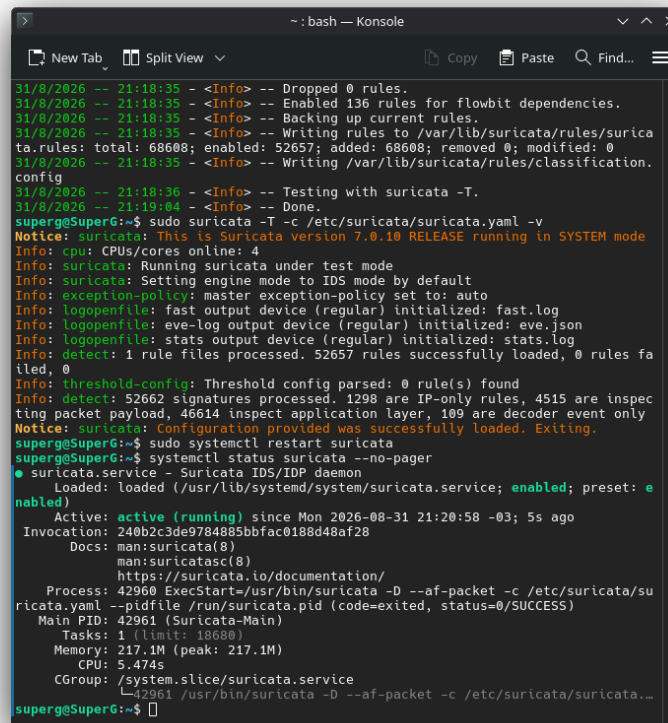
Figura 6 — Validación de configuración con 52.657 firmas cargadas

El desglose que ofrece la salida resulta ilustrativo del funcionamiento del motor: 1.298 firmas operan únicamente sobre direcciones IP, 4.515 inspeccionan la carga útil de los paquetes, 46.615 trabajan sobre protocolos de aplicación ya reconstruidos y 109 detectan paquetes mal formados a nivel de decodificación.

6. Puesta en servicio

6.1. Activación

```
sudo systemctl restart suricata
systemctl status suricata --no-pager
```



```
~: bash — Konsole
New Tab Split View Copy Paste Find...
31/8/2026 -- 21:18:35 - <Info> -- Dropped 0 rules.
31/8/2026 -- 21:18:35 - <Info> -- Enabled 136 rules for flowbit dependencies.
31/8/2026 -- 21:18:35 - <Info> -- Backing up current rules.
31/8/2026 -- 21:18:35 - <Info> -- Writing rules to /var/lib/suricata/rules/suricata.rules: total: 68608; enabled: 52657; added: 68608; removed 0; modified: 0
31/8/2026 -- 21:18:35 - <Info> -- Writing /var/lib/suricata/rules/classification.config
31/8/2026 -- 21:18:36 - <Info> -- Testing with suricata -T.
31/8/2026 -- 21:19:04 - <Info> -- Done.
superge@SuperG:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 4
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 1 rule files processed. 52657 rules successfully loaded, 0 rules failed, 0
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 52662 signatures processed. 1298 are IP-only rules, 4515 are inspecting packet payload, 46614 inspect application layer, 109 are decoder event only
Notice: suricata: Configuration provided was successfully loaded. Exiting.
superge@SuperG:~$ sudo systemctl restart suricata
superge@SuperG:~$ systemctl status suricata --no-pager
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-08-31 21:20:58 -03; 5s ago
     Invocation: 240b2c3de9784885bbfac0188d48af28
       Docs: man:suricata(8)
            man:suricatasc(8)
            https://suricata.io/documentation/
   Process: 42960 ExecStart=/usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid (code=exited, status=0/SUCCESS)
    Main PID: 42961 (Suricata-Main)
      Tasks: 1 (limit: 18680)
     Memory: 217.1M (peak: 217.1M)
        CPU: 5.474s
    CGroup: /system.slice/suricata.service
           └─42961 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata...
superge@SuperG:~$
```

Figura 7 — Servicio activo tras la corrección de la configuración

El servicio queda en estado activo y habilitado para el arranque automático. El consumo de memoria registrado al inicio ronda los 217 MB y crece durante los primeros minutos hasta estabilizarse cerca de 1 GB, comportamiento esperable dado el volumen de firmas que el motor mantiene en estructuras de búsqueda en memoria.

6.2. Verificación de captura

Un servicio en estado activo no garantiza que esté procesando paquetes. La comprobación se realiza sobre el registro de eventos en formato JSON.

```
sudo grep -o '"event_type": "[^"]*"' /var/log/suricata/eve.json \
| sort | uniq -c | sort -rn
```

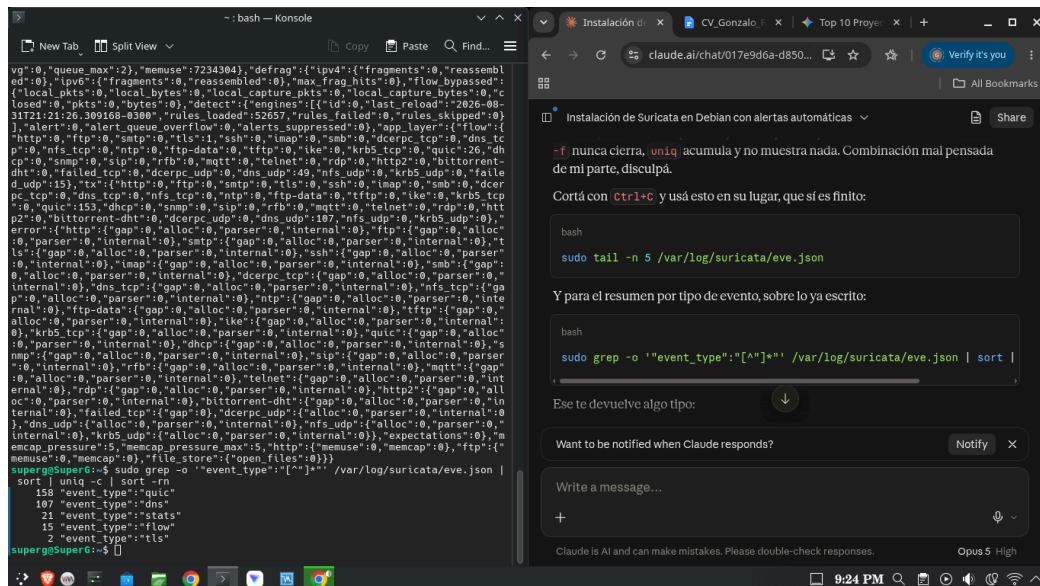


Figura 8 — Distribución de eventos capturados por tipo de protocolo

El recuento confirma captura efectiva y muestra un predominio marcado de eventos QUIC sobre los demás protocolos, seguidos de consultas DNS. Esa proporción refleja el estado actual del tráfico web y, como se señaló en el alcance, delimita hasta dónde puede inspeccionar un IDS de red sin terminación de cifrado.

Suricata genera tres registros con propósitos distintos. El archivo `fast.log` contiene únicamente alertas en texto plano, una por línea, y resulta cómodo para revisión rápida. El archivo `eve.json` registra la totalidad de los eventos en formato estructurado y es el que consumen las herramientas externas. El archivo `stats.log` acumula contadores de rendimiento del motor.

7. Reglas personalizadas

La prueba de detección prevista originalmente consistía en solicitar un recurso alojado en testmynids.org, sitio históricamente utilizado para validar sistemas de detección. Durante la ejecución se comprobó que el dominio ya no resuelve, lo que dejó la verificación sin efecto.

La alternativa adoptada resultó preferible: escribir firmas propias. Además de eliminar la dependencia de un recurso externo que puede desaparecer, permite documentar la sintaxis del lenguaje de reglas y las decisiones de diseño detrás de cada una.

7.1. Firma de detección por contenido

La primera regla busca una cadena identificatoria dentro de la URI de las peticiones HTTP.

```
alert http any any -> any any (msg:"LOCAL Prueba IDS - patron de  
test detectado"; http.uri; content:"SURICATA-TEST-GONZA";  
sid:1000001; rev:1;)
```



Figura 9 — Firma propia definida en el archivo local.rules

Cada elemento cumple una función definida. La acción alert indica que la coincidencia debe generar un evento sin bloquear el tráfico. El protocolo http restringe la inspección al análisis de ese protocolo ya reconstruido. Los pares any any delimitan origen y destino sin restricción de dirección ni puerto. El campo msg define el texto que aparecerá en el registro. El identificador sid debe ser único, y el rango a partir de 1000000 está reservado por convención para firmas locales, de modo que no colisiona con las de Emerging Threats.

Para que el motor considere el archivo hay que declararlo en la configuración, junto al conjunto principal.

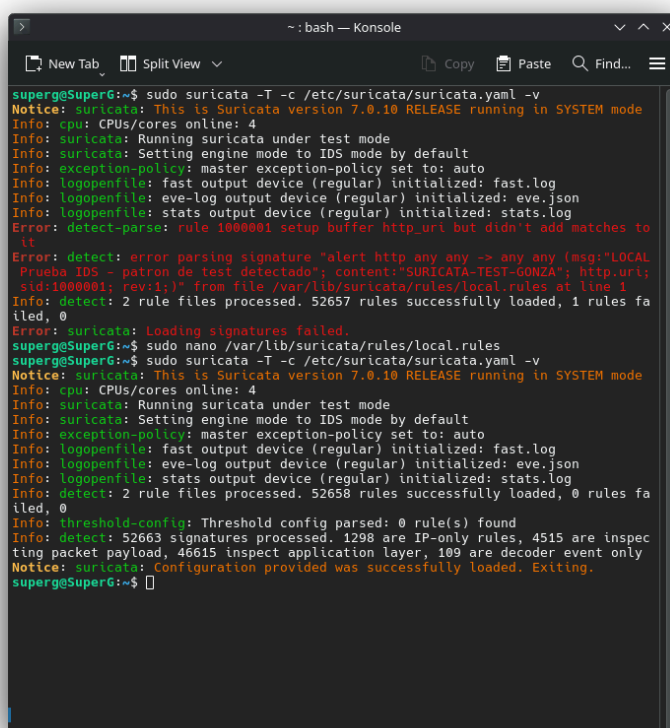
```
rule-files:  
- suricata.rules  
- local.rules
```

7.2. Error de parseo y corrección

La primera validación tras agregar la firma falló con un mensaje que indicaba que la regla declaraba el buffer `http_uri` sin añadirle coincidencias. El orden original de la firma colocaba el modificador `http.uri` después de la palabra clave `content`, siguiendo la sintaxis heredada de Snort.

Suricata 7 utiliza buffers de tipo sticky, cuyo comportamiento es el inverso. La declaración del buffer no modifica lo que la precede, sino que establece el contexto de inspección para todo lo que viene a continuación. Con el orden invertido, `content` buscaba sobre la carga útil sin procesar mientras el buffer quedaba declarado y vacío, condición que el analizador rechaza.

La corrección consiste en anteponer la declaración del buffer a la cadena buscada, tal como figura en la regla presentada en el apartado anterior.



```
~: bash — Konsole
New Tab Split View Copy Paste Find...
superG@SuperG:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 4
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Error: detect-parse: rule 1000001 setup buffer http.uri but didn't add matches to it
Error: detect: error parsing signature "alert http any any -> any any [msg:"LOCAL
Prueba IDS - patron de test detectado"; content:"SURICATA-TEST-GONZA": http.uri;
sid:1000001; rev:1]" from file /var/lib/suricata/rules/local.rules at line 1
Info: detect: 2 rule files processed. 52657 rules successfully loaded, 1 rules fa
iled, 0
Error: suricata: Loading signatures failed.
superG@SuperG:~$ sudo nano /var/lib/suricata/rules/local.rules
superG@SuperG:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 7.0.10 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 4
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 2 rule files processed. 52658 rules successfully loaded, 0 rules fa
iled, 0
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 52663 signatures processed. 1298 are IP-only rules, 4515 are inspec
ting packet payload, 46615 inspect application layer, 109 are decoder event only
Notice: suricata: Configuration provided was successfully loaded. Exiting.
superG@SuperG:~$
```

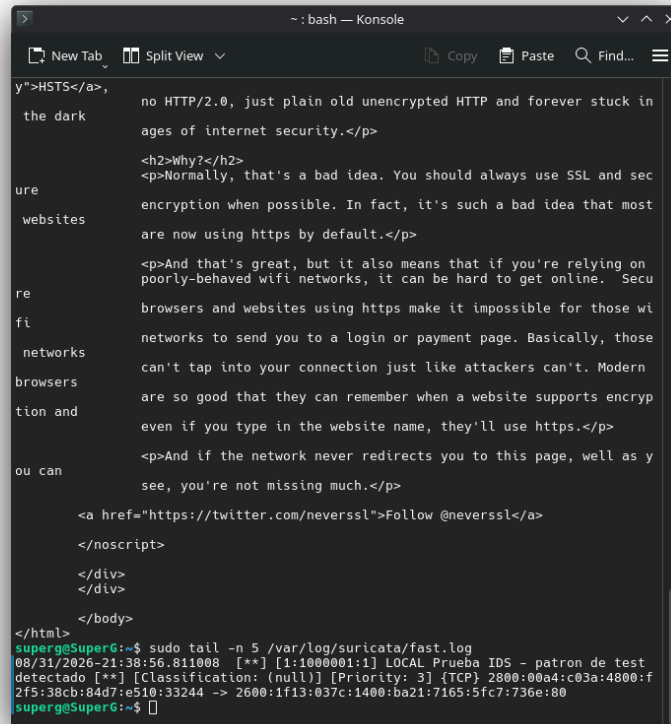
Figura 10 — Error de parseo y validación posterior a la corrección

La captura muestra ambos momentos en la misma pantalla: el rechazo inicial con una firma fallida y la validación posterior con 52.658 reglas cargadas y ninguna fallida.

7.3. Verificación de la detección

Para disparar la firma se solicitó un recurso HTTP incluyendo la cadena buscada en la URI. Se recurrió a neverssl.com por tratarse de un sitio que se mantiene deliberadamente sin cifrado; sobre HTTPS la URI resultaría inaccesible para el motor y la regla no coincidiría.

```
curl -s "http://neverssl.com/?test=SURICATA-TEST-GONZA" > /dev/null
sudo tail -n 5 /var/log/suricata/fast.log
```



```
~: bash — Konsole
New Tab Split View Copy Paste Find...
y">HSTS</a>,
the dark    no HTTP/2.0, just plain old unencrypted HTTP and forever stuck in
            ages of internet security.</p>
            <h2>Why?</h2>
            <p>Normally, that's a bad idea. You should always use SSL and sec
ure         encryption when possible. In fact, it's such a bad idea that most
websites    are now using https by default.</p>
            <p>And that's great, but it also means that if you're relying on
re          poorly-behaved wifi networks, it can be hard to get online. Secu
fi          browsers and websites using https make it impossible for those wi
networks    networks to send you to a login or payment page. Basically, those
browsers    can't tap into your connection just like attackers can't. Modern
tion and    are so good that they can remember when a website supports encryp
ou can      even if you type in the website name, they'll use https.</p>
            <p>And if the network never redirects you to this page, well as y
see, you're not missing much.</p>
            <a href="https://twitter.com/neverssl">Follow @neverssl</a>
            </noscript>
            </div>
            </div>
        </body>
    </html>
superge@Super6:~$ sudo tail -n 5 /var/log/suricata/fast.log
08/31/2026-21:38:56.811000 [**] [1:1000001:1] LOCAL Prueba IDS - patron de test
detectado [**] [Classification: (null)] [Priority: 3] (TCP) 2800:00a4:c03a:4000:f
2f5:38cb:84d7:e510:33244 -> 2600:1f13:037c:1400:ba21:7165:5fc7:736e:80
superge@Super6:~$
```

Figura 11 — Primera alerta generada por una firma propia

La entrada registrada contiene el identificador 1:1000001:1, el mensaje definido en la firma y las direcciones involucradas. Cabe observar que estas últimas son IPv6, lo que confirma que el motor inspecciona ambas pilas de protocolo sin configuración adicional.

7.4. Firma de detección de escaneo

Una primera prueba con nmap contra el router de la red no produjo alerta alguna. La causa remite nuevamente a la variable HOME_NET: tanto el origen como el destino del escaneo pertenecen al segmento protegido, mientras que las firmas de reconocimiento de Emerging Threats se redactan desde la red externa hacia la interna. A esto se suma que buena parte de las reglas de la categoría de escaneo vienen deshabilitadas por defecto debido al ruido que generan.

La solución fue escribir una firma con una lógica distinta a la anterior: en lugar de buscar un contenido, detecta un patrón de comportamiento mediante un umbral.

```
alert tcp any any -> $HOME_NET any (msg:"LOCAL Posible escaneo de
puertos detectado"; flags:S; threshold:type both, track by_src,
count 30, seconds 10; sid:1000002; rev:1;)
```

El modificador flags:S restringe la coincidencia a paquetes con únicamente el bit SYN activo, que es la firma característica del escaneo de media apertura. La cláusula threshold introduce la dimensión temporal: agrupa los paquetes por dirección de origen y exige treinta ocurrencias en diez segundos antes de emitir un evento. El parámetro type both cumple además una función de contención, ya que limita la emisión a una alerta por ventana.

Esa última decisión resulta determinante para la usabilidad del sistema. Un escaneo de mil puertos genera mil paquetes; sin umbral, el usuario recibiría mil notificaciones y el escritorio quedaría inutilizable. El umbral convierte la detección por paquete en detección por patrón, que es lo que interesa desde el punto de vista analítico.

8. Notificación de alertas en el escritorio

8.1. Consideración arquitectónica

El requisito de mostrar alertas en pantalla plantea un obstáculo que no es de configuración sino de arquitectura. Suricata se ejecuta como servicio de sistema bajo el usuario root, y los procesos de root no tienen acceso al bus de sesión del entorno gráfico. Cualquier intento de emitir una notificación desde ese contexto se pierde sin generar error.

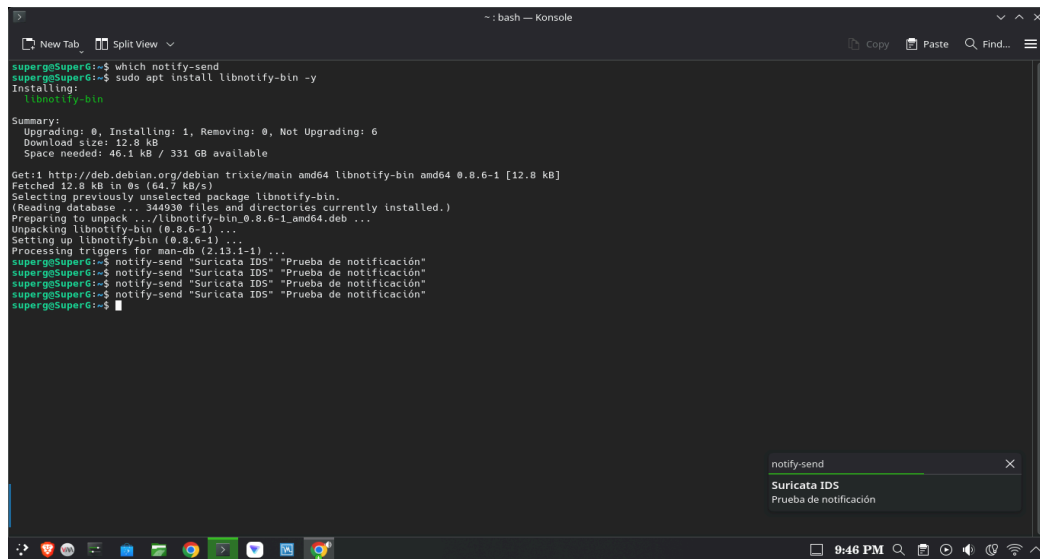
La solución adoptada separa responsabilidades en dos componentes. El motor conserva sus privilegios de sistema, necesarios para la captura en modo promiscuo, y escribe sus eventos en disco. Un segundo proceso, ejecutado dentro de la sesión del usuario, lee ese archivo y traduce cada alerta en una notificación. El archivo de eventos actúa como punto de contacto entre ambos, condición viable porque sus permisos lo dejan legible para todos los usuarios.

```
Interfaz de red
|
v
suricata.service (root, systemd de sistema)
|
v
/var/log/suricata/eve.json (permisos 644)
|
v
suricata-notify.service (usuario, systemd --user)
|
v
notify-send -> KDE Plasma
```

8.2. Verificación del canal de notificaciones

Antes de construir el componente conviene comprobar de forma aislada que el mecanismo de notificaciones responde.

```
sudo apt install libnotify-bin -y
notify-send "Suricata IDS" "Prueba de notificación"
```

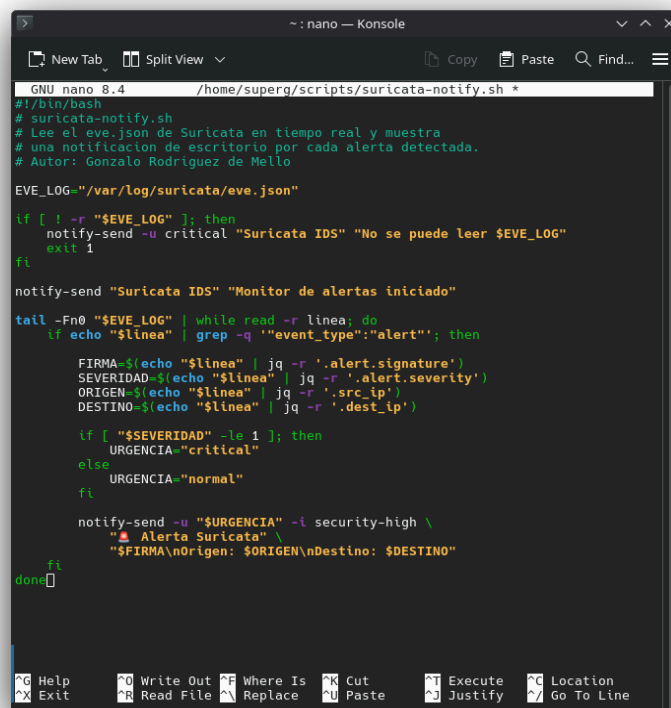


```
~: bash — Konsole
New Tab Split View
Copy Paste Find...
superg@superg:~$ which notify-send
superg@superg:~$ sudo apt install libnotify-bin -y
Installing:
libnotify-bin
Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 6
Download size: 12.8 KB
Space needed: 46.1 KB / 331 GB available
Get:1 http://deb.debian.org/debian trixie/main amd64 libnotify-bin amd64 0.8.6-1 [12.8 KB]
Fetched 12.8 KB in 0s (64.7 KB/s)
Selecting previously unselected package libnotify-bin.
(Reading database ... 348930 files and directories currently installed.)
Preparing to unpack .../libnotify-bin_0.8.6-1_amd64.deb ...
Unpacking libnotify-bin (0.8.6-1) ...
Setting up libnotify-bin (0.8.6-1) ...
Processing triggers for man-db (2.13.1-1) ...
superg@superg:~$ notify-send "Suricata IDS" "Prueba de notificación"
superg@superg:~$ notify-send "Suricata IDS" "Prueba de notificación"
superg@superg:~$ notify-send "Suricata IDS" "Prueba de notificación"
superg@superg:~$ notify-send "Suricata IDS" "Prueba de notificación"
superg@superg:~$
```

Figura 12 — Comprobación aislada del canal de notificaciones de Plasma

8.3. Script de vigilancia

El componente intermedio es un script en Bash que sigue el archivo de eventos en tiempo real, filtra las líneas correspondientes a alertas, extrae los campos relevantes y emite la notificación.



```
~: nano — Konsole
New Tab Split View
Copy Paste Find...
GNU nano 8.4 /home/superg/scripts/suricata-notify.sh *
#!/bin/bash
# suricata-notify.sh
# Lee el eve.json de Suricata en tiempo real y muestra
# una notificación de escritorio por cada alerta detectada.
# Autor: Gonzalo Rodriguez de Mello

EVE_LOG="/var/log/suricata/eve.json"

if [ ! -r "$EVE_LOG" ]; then
    notify-send -u critical "Suricata IDS" "No se puede leer $EVE_LOG"
    exit 1
fi

notify-send "Suricata IDS" "Monitor de alertas iniciado"

tail -Fn0 "$EVE_LOG" | while read -r linea; do
    if echo "$linea" | grep -q "event_type\":\"alert\""; then
        FIRMA=$(echo "$linea" | jq -r '.alert.signature')
        SEVERIDAD=$(echo "$linea" | jq -r '.alert.severity')
        ORIGEN=$(echo "$linea" | jq -r '.src_ip')
        DESTINO=$(echo "$linea" | jq -r '.dest_ip')

        if [ "$SEVERIDAD" -le 1 ]; then
            URGENCIA="critical"
        else
            URGENCIA="normal"
        fi

        notify-send -u "$URGENCIA" -i security-high \
            "Alerta Suricata" \
            "$FIRMA\nOrigen: $ORIGEN\nDestino: $DESTINO"
    fi
done
```

Figura 13 — Código del script de vigilancia sobre el registro de eventos

Las decisiones de implementación merecen comentario. La opción -F de tail mantiene el seguimiento aunque el archivo sea rotado, situación que ocurre a diario por acción de logrotate; con la opción -f convencional el script quedaría siguiendo un descriptor huérfano y dejaría de reportar sin dar aviso. El parámetro -n0 evita que se reprocese el historial acumulado al iniciar.

El filtrado se realiza en dos etapas por razones de rendimiento. Un grep descarta primero las líneas que no corresponden a alertas, que constituyen la enorme mayoría del archivo, y solo las restantes se entregan a jq. Invocar el intérprete de JSON sobre cada evento de flujo o consulta DNS resultaría costoso sin beneficio alguno.

La extracción de campos se delega en jq y no en expresiones regulares. El registro de eventos contiene cadenas con comillas escapadas y estructuras anidadas que un procesamiento textual maneja de forma poco confiable. Finalmente, el nivel de severidad determina la urgencia de la notificación: las de nivel uno permanecen en pantalla hasta que el usuario las descarta, mientras que las restantes se ocultan de forma automática.

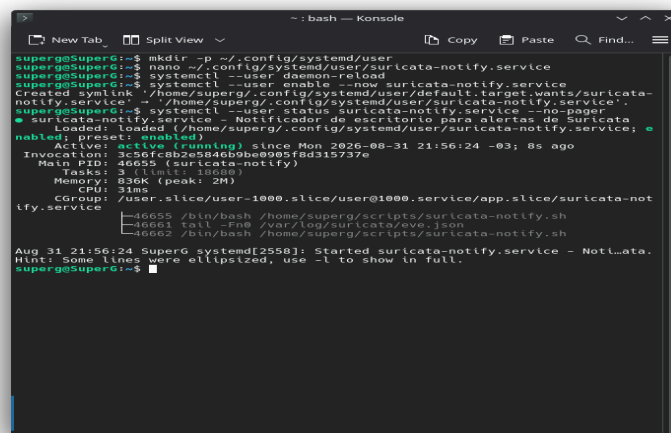
8.4. Servicio de usuario

Para que el script se ejecute de forma persistente sin depender de una terminal abierta se definió una unidad de systemd en el ámbito del usuario, no del sistema. Esta distinción es la que garantiza el acceso al bus de sesión gráfica.

```
[Unit]
Description=Notificador de escritorio para alertas de Suricata
After=graphical-session.target
PartOf=graphical-session.target

[Service]
Type=simple
ExecStart=%h/scripts/suricata-notify.sh
Restart=on-failure
RestartSec=5

[Install]
WantedBy=default.target
systemctl --user daemon-reload
systemctl --user enable --now suricata-notify.service
```



```
super@SuperG:~$ mkdir -p ~/.config/systemd/user
super@SuperG:~$ nano ~/.config/systemd/user/suricata-notify.service
super@SuperG:~$ systemctl --user daemon-reload
super@SuperG:~$ systemctl --user enable --now suricata-notify.service
Created symlink /home/superg/.config/systemd/user/default.target.wants/suricata-notify.service' -> /home/superg/.config/systemd/user/suricata-notify.service'.
super@SuperG:~$ systemctl --user status suricata-notify.service --no-pager
● suricata-notify.service - Notificador de escritorio para alertas de Suricata
   Loaded: loaded (/home/superg/.config/systemd/user/suricata-notify.service; e
nabled; preset: enabled)
   Active: active (running) since Mon 2026-08-31 21:56:24 -03; 8s ago
     Invocation: 3c56fc8b2e5d4d0bae0995f8d315737e
    Main PID: 46655 (suricata-notify)
       Tasks: 3 (1 shell, 1 idle)
      Memory: 836K (peak: 2M)
         CPU: 31ms
    CGroup: /user.slice/user-1000.slice/user@1000.service/app.slice/suricata-not
ify.service
            └─46655 /bin/bash /home/superg/scripts/suricata-notify.sh
                  └─46661 tail -Fno /var/log/suricata/eve.json
                  └─46662 /bin/bash /home/superg/scripts/suricata-notify.sh

Aug 31 21:56:24 SuperG systemd[2558]: Started suricata-notify.service - Notif.ata.
super@SuperG:~$
```

Figura 14 — Unidad de usuario activa con su árbol de procesos

El árbol de control confirma la ubicación correcta del servicio dentro de user@1000.service, junto a los tres procesos que lo componen: el script principal, el proceso de seguimiento del archivo y el subshell que procesa cada línea. El consumo de memoria se mantiene por debajo del megabyte.

8.5. Verificación funcional

Con ambos componentes en ejecución se repitió la petición HTTP que activa la primera firma.

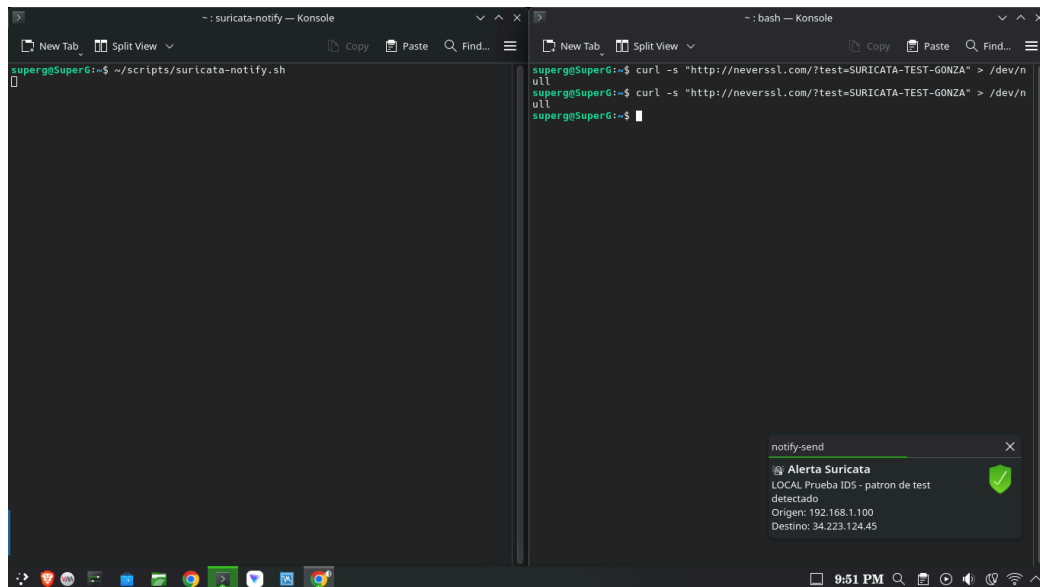


Figura 15 — Cadena completa en funcionamiento, del paquete a la notificación

La notificación presenta el nombre de la firma junto al origen y el destino del tráfico. En esta ocasión las direcciones son IPv4 y el origen corresponde a la dirección del equipo dentro del segmento configurado, lo que confirma de manera indirecta que la definición de la red protegida es correcta.

9. Prueba integral

La validación final consistió en ejecutar un escaneo de puertos de media apertura contra el router de la propia red, dirigido a comprobar la firma basada en umbral y, con ella, el recorrido completo desde la captura hasta la notificación.

```
sudo nmap -sS -p 1-1000 192.168.1.1
```

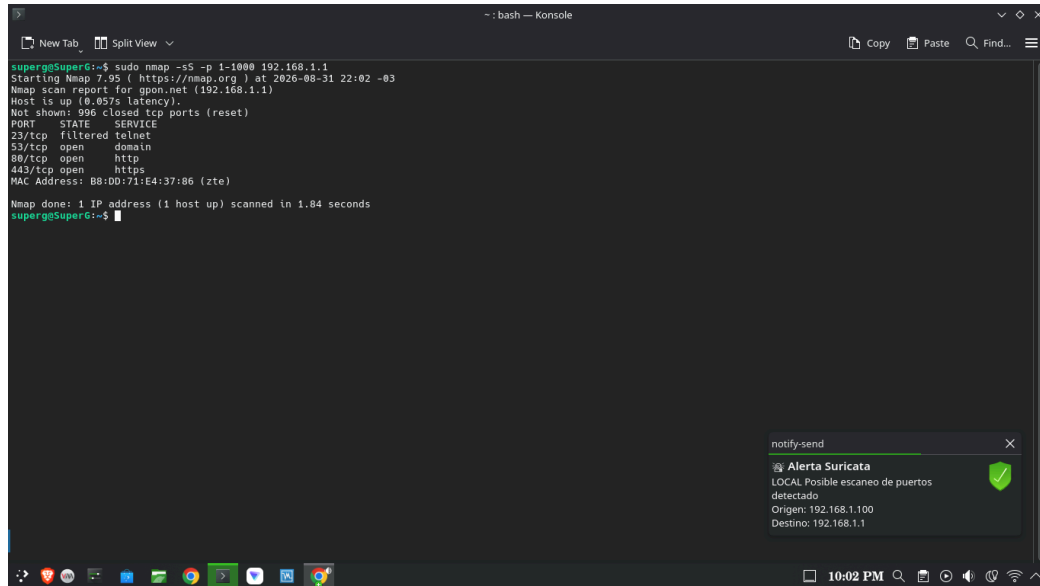


Figura 16 — Escaneo de puertos detectado y notificado en tiempo real

El escaneo se completó en menos de dos segundos y el sistema emitió una única alerta, comportamiento esperado según el umbral definido. El registro consigna el identificador 1:1000002:1 junto a las direcciones de origen y destino. El puerto de destino que figura en la entrada no reviste significado particular: corresponde simplemente al paquete en el cual se alcanzó el umbral configurado.

Corresponde señalar que este tipo de prueba debe realizarse exclusivamente sobre infraestructura propia. El escaneo de sistemas ajenos sin autorización expresa constituye una conducta tipificada por la normativa sobre delitos informáticos.

10. Mantenimiento

El conjunto de firmas requiere actualización periódica, ya que su valor depende de que refleje las amenazas en circulación. La operación se resuelve con dos comandos que conviene ejecutar de forma programada.

```
sudo suricata-update  
sudo systemctl restart suricata
```

Cualquier modificación de la configuración o del conjunto de reglas debe validarse antes de reiniciar el servicio, para evitar que un error de sintaxis deje el motor fuera de línea.

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v  
systemctl --user status suricata-notify.service
```

11. Conclusiones

El sistema quedó operativo con 52.659 firmas activas, dos de ellas de elaboración propia, y notifica en pantalla dentro de los segundos posteriores a la detección. Ambos servicios se inician de forma automática, el del motor con el sistema y el del notificador con la sesión gráfica, sin requerir intervención del usuario.

Cuatro conceptos concentran el aprendizaje del despliegue. El primero es que la variable HOME_NET determina la orientación de la totalidad del conjunto de reglas, y una definición incorrecta produce un sistema aparentemente sano que no detecta nada. El segundo es el funcionamiento de los buffers sticky, que invierte el orden de escritura respecto de la sintaxis heredada. El tercero es el papel del umbral, que transforma la detección por paquete en detección por patrón y hace viable la notificación al usuario. El cuarto es la separación de contextos entre el servicio de sistema y el de sesión, condición sin la cual las notificaciones nunca alcanzan el escritorio.

11.1. Líneas de trabajo posteriores

El registro de eventos en formato JSON constituye el punto natural de extensión del sistema. Su reenvío hacia una plataforma de gestión de eventos permitiría correlacionar las detecciones con registros de otras fuentes, aplicar reglas de agregación y conservar un historial consultable, capacidades que el archivo plano no ofrece.

Sobre esa misma base resultan posibles otras extensiones: la incorporación de reglas de respuesta activa que ejecuten acciones ante determinadas firmas, y la ampliación del alcance de la captura a la interfaz virtual del hipervisor, de modo de inspeccionar también el tráfico de las máquinas virtuales del equipo.

Anexo. Inventario de archivos

Los componentes que integran el despliegue se distribuyen de la siguiente manera.

- `/etc/suricata/suricata.yaml` — configuración principal del motor.
- `/var/lib/suricata/rules/suricata.rules` — conjunto Emerging Threats Open.
- `/var/lib/suricata/rules/local.rules` — firmas de elaboración propia.
- `/var/log/suricata/eve.json` — registro estructurado de eventos.
- `/var/log/suricata/fast.log` — registro de alertas en texto plano.
- `~/scripts/suricata-notify.sh` — script de vigilancia y notificación.
- `~/.config/systemd/user/suricata-notify.service` — unidad de usuario.